Getting Started with LangChain 1.0+

O'Reilly Live Training — Build AI Agents with LangChain & LangGraph

Python 3.11–3.13 | LangChain 1.0+ | LangGraph 1.0+ | By Lucas Soares

COURSE AT A GLANCE

00	1	2	3	★
Agents from Scratch	Fundamentals	Structured Output	RAG	Demos

Quick Setup

```
git clone <repo>
cp .env.example .env   # add API keys
uv sync
uv run jupyter lab
make demo              # start agent server
```

Required API Keys (.env)

- `OPENAI_API_KEY` — LLM calls
- `TAVILY_API_KEY` — web search tools
- `LANGSMITH_API_KEY` — LangSmith tracing (optional)
- `LANGSMITH_TRACING=true` — enable tracing

MODULE 1 — THE 6 BUILDING BLOCKS

1. Agents — `create_agent()` — The Central API

`create_agent()` signature

```
from langchain.agents import create_agent

agent = create_agent(
    model,            # "provider:model" or
ChatModel
    tools=None,       # list of @tool functions
    system_prompt=None,
    checkpointer=None,  # for memory
    response_format=None,  # for structured output
)

# Invoke
result = agent.invoke({"messages": "Hello"})
result["messages"][-1].pretty_print()

# Minimal (3 lines)
agent = create_agent(model="openai:gpt-6-luna",
tools=[])
result = agent.invoke({"messages": "What is
LangChain?"})
```

Agent with Tools

```
from langchain_tavily import TavilySearch

search = TavilySearch(max_results=5)
# Reasoning models need reasoning_effort="none" to
use function tools
model = init_chat_model("openai:gpt-6-luna",
reasoning_effort="none")

agent = create_agent(
    model=model,
    tools=[search],
    system_prompt="You are a helpful assistant.",
)

# Stream agent steps
for step in agent.stream(
    {"messages": "What's new in AI?"},
    stream_mode="values",
):
    step["messages"][-1].pretty_print()
    print("---")
```

2. Models — `init_chat_model()` — Universal Factory

`init_chat_model()` — 3 invocation modes

```
from langchain.chat_models import init_chat_model

model = init_chat_model("openai:gpt-6-luna")

# invoke() — single response
r = model.invoke("Why is the sky blue?")
print(r.content)

# stream() — token by token
for chunk in model.stream("Tell me a joke"):
    print(chunk.content, end="", flush=True)

# batch() — parallel requests
responses = model.batch(["Q1?", "Q2?", "Q3?"])
for r in responses:
    print(r.content)
```

Provider String Formats

Provider	String Format	Package
OpenAI	`"openai:gpt-6-luna"`	langchain-openai
Anthropic	`"anthropic:claude-haiku-5-5"`	langchain-anthropic
Google	`"google_genai:gemini-3.5-flash-lite"`	langchain-google-genai
Ollama	`"ollama:gemma4"`	langchain-ollama

3. Messages — 4 Types

Message Types

Type	Role	Purpose
`SystemMessage`	system	Sets model behavior/persona
`HumanMessage`	user	User input
`AIMessage`	assistant	Model response
`ToolMessage`	tool	Tool execution result

Messages usage

```
from langchain_core.messages import (
    SystemMessage, HumanMessage, AIMessage
)

# Object format
msgs = [
    SystemMessage("You are a pirate."),
    HumanMessage("What is ML?"),
]
r = model.invoke(msgs)

# Dict format (OpenAI-compatible)
r = model.invoke([
    {"role": "system", "content": "..."},
    {"role": "user", "content": "..."},
])

# Multimodal (image)
msg = HumanMessage(content=[
    {"type": "text", "text": "Describe this."},
    {"type": "image_url",
     "image_url": {"url": "https://..."}},
])
```

4. Tools — `@tool` Decorator — Two Mandatory Rules: Type Hints + Docstrings

Basic @tool + Pydantic schema

```
from langchain_core.tools import tool
from pydantic import BaseModel, Field
from typing import Literal

@tool
def calculate(expression: str) → str:
    """Evaluate a math expression like '2 + 2'."""
    return str(eval(expression))

# Pydantic for complex inputs
class WeatherQuery(BaseModel):
    location: str = Field(description="City name")
    units: Literal["celsius","fahrenheit"] =
"celsius"

@tool(args_schema=WeatherQuery)
def get_weather(location: str, units: str) → str:
    """Get current weather for a location."""
    return f"Weather in {location}: 22 C, Sunny"

# Check tool schema
print(calculate.name)
print(calculate.description)
```

Dependency injection with ToolRuntime

```
from dataclasses import dataclass
from langchain.tools import tool, ToolRuntime

@dataclass
class UserContext:
    user_name: str
    role: str

USER_DB = {
    "alice": {"balance": 5000, "plan": "premium"},
}

@tool
def get_account_info(
    runtime: ToolRuntime[UserContext]
) → str:
    """Get the current user's account info."""
    name = runtime.context.user_name
    info = USER_DB.get(name, {})
    return (f"User: {name} | "
            f"Balance: ${info.get('balance')}")

# context injected at runtime — not exposed to LLM
# agent = create_agent(...,
context_schema=UserContext)
```

5. Short-Term Memory — `InMemorySaver + thread_id`

Memory with checkpointer

```
from langgraph.checkpoint.memory import
InMemorySaver

model = init_chat_model("openai:gpt-6-luna",
reasoning_effort="none")
agent = create_agent(
    model=model,
    tools=[search],
    checkpointer=InMemorySaver(),
)

config = {"configurable": {"thread_id":
"session-1"}}

# Turn 1 — introduce yourself
agent.invoke(
    {"messages": "My name is Lucas."},
    config,
)

# Turn 2 — agent remembers!
result = agent.invoke(
    {"messages": "What is my name?"},
    config,         # same thread_id = same memory
)
# "Your name is Lucas."

# Fresh memory with new thread_id
config2 = {"configurable": {"thread_id":
"session-2"}}
# no knowledge of Lucas
```

Memory Key Facts

- **Thread isolation:** each `thread_id` is fully independent
- **InMemorySaver:** in-process only, resets on restart
- **Persistent memory:** use `SqliteSaver` or `PostgresSaver` for production
- **Config is required:** pass `config` as 2nd arg to `invoke()` / `stream()`
- **LangGraph dev:** provides persistence automatically — no checkpointer needed in demo

6. Streaming — 3 Modes

Stream Mode Comparison

Mode	Output	Best For
`"values"`	Full state after each step	Debug / track reasoning
`"messages"`	`(token, metadata)` tuples	Real-time chatbot UX
`"custom"`	User-defined data	Progress updates from tools

All 3 stream modes

```
# Mode 1: values
for step in agent.stream(
    {"messages": "..."}, stream_mode="values"
):
    step["messages"][-1].pretty_print()

# Mode 2: messages (chatbot UX)
for token, _ in agent.stream(
    {"messages": "..."}, stream_mode="messages"
):
    if token.content:
        print(token.content, end="", flush=True)

# Mode 3: custom (tool progress)
from langgraph.config import get_stream_writer

@tool
def long_task(query: str) → str:
    """Task with progress updates."""
    writer = get_stream_writer()
    writer({"status": "Searching..."})
    return "Done"

# combine modes
for chunk in agent.stream(
    {"messages": "..."},
    stream_mode=["values", "custom"],
):
    print(chunk)
```

MODULE 2 — STRUCTURED OUTPUTS & PRACTICAL APPLICATIONS

Approach 1: TypedDict (returns dict)

```
from typing_extensions import TypedDict
from langchain.agents import create_agent

class ContactInfo(TypedDict):
    name: str
    email: str
    phone: str

agent = create_agent(
    model="openai:gpt-6-luna",
    response_format=ContactInfo,
)

result = agent.invoke({
    "messages": "Hi, I'm Jane, jane@ex.com,
(555)123"
})
contact = result["structured_response"]  # dict
print(contact["name"])   # "Jane"
```

Approach 2: Pydantic BaseModel (recommended)

```
from pydantic import BaseModel, Field
from typing import List, Literal

class MovieReview(BaseModel):
    title: str = Field(description="Movie title")
    sentiment:
Literal["positive","negative","mixed"]
    rating: int = Field(ge=1, le=10)
    key_points: List[str]
    recommended: bool

agent = create_agent(
    model="openai:gpt-6-luna",
    response_format=MovieReview,
)
result = agent.invoke({
    "messages": "Review: Inception is amazing, 9/10"
})
review = result["structured_response"]  #
MovieReview
print(review.title, review.rating)  # Inception 9
```

Strategy Selection

Strategy	How it works	Best for
`ProviderStrategy`	Native structured output API	OpenAI, Anthropic, Gemini
`ToolStrategy`	Uses tool-calling mechanism	Any model with tool support
Auto (pass schema)	LangChain picks best	Getting started

Explicit strategy selection

```
from langchain.agents.structured_output import (
    ProviderStrategy, ToolStrategy
)

# ToolStrategy uses function-tool calling under the
hood,
# which requires reasoning_effort="none" on this
endpoint.
model = init_chat_model("openai:gpt-6-luna",
reasoning_effort="none")

# Native (fastest for OpenAI/Anthropic)
agent = create_agent(
    model=model,

response_format=ProviderStrategy(schema=MovieReview)
,
)

# Tool-call based (universal fallback)
agent = create_agent(
    model=model,

response_format=ToolStrategy(schema=MovieReview),
)

# Both return: result["structured_response"]
```

Practical Application: Resume Analyzer & Job-Fit Scorer

Nested Pydantic models

```
class SkillAssessment(BaseModel):
    skill_name: str
    proficiency: Literal[
        "expert","proficient","familiar","none"]
    evidence: str

class CandidateProfile(BaseModel):
    name: str
    years_experience: int
    current_role: str
    top_skills: List[str]

class JobFitEvaluation(BaseModel):
    candidate: CandidateProfile
    skill_assessments: List[SkillAssessment]
    fit_score: int = Field(ge=0, le=100)
    strengths: List[str]
    gaps: List[str]
    recommendation: Literal[
        "strong_hire","hire","maybe","no_hire"]
    summary: str
```

Using structured output for batch comparison

```
resume_agent = create_agent(
    model="openai:gpt-6-luna",
    system_prompt="You are a technical recruiter.",
    response_format=JobFitEvaluation,
)

result = resume_agent.invoke({
    "messages": f"Evaluate:\nJOB:\n{job}\n\n"
                f"RESUME:\n{resume}"
})
ev = result["structured_response"]

# Access structured fields
print(ev.candidate.name)
print(ev.fit_score)          # 85
print(ev.recommendation)     # "hire"
print(ev.gaps)               # ["Missing
Kubernetes"]

# Batch comparison only possible with structured
output!
for ev in sorted(evals,
    key=lambda x: x.fit_score, reverse=True):
    print(f"{ev.candidate.name}: {ev.fit_score}%")
```

MODULE 3 — RAG (RETRIEVAL AUGMENTED GENERATION)

Part 1: Indexing Pipeline — Load → Split → Embed → Store

1	2	3	4
Load	**Split**	**Embed**	**Store**
`WebBaseLoader` or `PyPDFLoader`	`RecursiveCharacterTextSplitter` chunk_size=1000	`OpenAIEmbeddings` text to vectors	`Chroma` vector database

Complete indexing pipeline

```
from langchain_community.document_loaders import WebBaseLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_chroma import Chroma
from langchain_openai import OpenAIEmbeddings

# 1. Load
loader = WebBaseLoader(["https://example.com/article"])
docs = loader.load()

# 2. Split
splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=200)
splits = splitter.split_documents(docs)

# 3+4. Embed and Store
vector_store = Chroma.from_documents(
    documents=splits,
    embedding=OpenAIEmbeddings(model="text-embedding-3-small"),
    collection_name="my-docs",
    persist_directory="./chroma-db"        # omit for in-memory
)

# Retrieve
retriever = vector_store.as_retriever(search_kwargs={"k": 4})
docs = retriever.invoke("what is task decomposition?")
```

Part 2: RAG Agents (flexible) vs RAG Chains (fast)

RAG Agent — LLM decides when to search

```
from langchain_core.tools import tool
from langchain.agents import create_agent

@tool(response_format="content_and_artifact")
def retrieve_context(query: str):
    """Retrieve information to help answer a
query."""
    docs = vector_store.similarity_search(query,
k=2)
    serialized = "\n\n".join(
        f"Source: {d.metadata}\n{d.page_content}"
        for d in docs
    )
    return serialized, docs   # str + list[Document]

agent = create_agent(
    model,
    [retrieve_context],
    system_prompt=(
        "Search documents before answering. "
        "Cite sources."
    ),
)

result = agent.invoke({
    "messages": [{"role":"user","content":"What is
RAG?"}]
})
```

RAG Chain — always searches, single LLM call

```
from langchain.agents.middleware import (
    dynamic_prompt, ModelRequest,
    AgentMiddleware, AgentState
)
from langchain_core.documents import Document
from typing import Any

# Simple: @dynamic_prompt
@dynamic_prompt
def prompt_with_context(request: ModelRequest) →
str:
    query = request.state["messages"][-1].text
    docs = vector_store.similarity_search(query)
    ctx = "\n\n".join(d.page_content for d in docs)
    return f"Use this context:\n\n{ctx}"

chain = create_agent(
    model, tools=[],
middleware=[prompt_with_context])

# Advanced: AgentMiddleware (exposes source docs)
class RetrieveDocs(AgentMiddleware[AgentState]):
    def before_model(self, state):
        msg = state["messages"][-1]
        docs =
vector_store.similarity_search(msg.text)
        ctx = "\n\n".join(d.page_content for d in
docs)
        return {
            "messages": [msg.model_copy(update={
                "content": f"{msg.text}\n\nContext:
\n{ctx}"
            })],
            "context": docs,
        }

chain2 = create_agent(
    model, tools=[], middleware=[RetrieveDocs()])
```

RAG Agent vs RAG Chain

	RAG Agent	RAG Chain
LLM calls	2 per query	1 per query
Search control	LLM decides when to search	Always searches first
Multiple searches	Yes (for complex queries)	No (single retrieval)
Flexibility	High — can skip search	Low — always retrieves
Best for	Complex, open-ended queries	Simple, fast, predictable queries

DEMO — DEPLOYABLE CHAT-OVER-DOCS AGENT

demo/agent.py — production RAG pattern

```
from langchain.agents import create_agent
from langchain.chat_models import init_chat_model
from langchain_core.tools import tool
from langchain_tavily import TavilySearch

def _build_retriever():
    docs = WebBaseLoader(URLS).load()
    splits = RecursiveCharacterTextSplitter(
        chunk_size=1000, chunk_overlap=200
    ).split_documents(docs)
    vs = Chroma.from_documents(
        documents=splits,
        embedding=OpenAIEmbeddings(
            model="text-embedding-3-small"),
        collection_name="course_docs",
    )
    return vs.as_retriever(search_kwargs={"k": 4})

# Eager: built once, at import time
_retriever = _build_retriever()

@tool
def search_documents(query: str) → str:
    """Search the loaded documents."""
    docs = _retriever.invoke(query)
    return "\n\n".join(
        f"[Source: {d.metadata.get('source')}]\n"
        f"{d.page_content}" for d in docs
    )

search = TavilySearch(max_results=5)  # web

agent = create_agent(
    model=init_chat_model(
        "openai:gpt-6-luna", reasoning_effort="none"
    ),
    tools=[search_documents, search],
    system_prompt=SYSTEM_PROMPT,
    # No checkpointer -- langgraph dev persists
)
```

Deploy & Connect

Start server:

```
make demo
# OR: uv run langgraph dev
```

Connect UI:

1. Visit **agentchat.vercel.app**
2. Connect to `http://localhost:2024`
3. Graph ID: `chat_over_docs`

langgraph.json — deployment config

LangSmith auto-traces all runs when `LANGSMITH_TRACING=true`

Key pattern: eager init — `_retriever = _build_retriever()` builds the vector store once at import time; `TavilySearch` answers off-topic questions

QUICK REFERENCE — KEY APIS

Complete API Reference

Concept	API / Pattern	Key Parameter / Note
Create agent	`create_agent(model, tools)`	`checkpointer`, `response_format`, `system_prompt`
Init model	`init_chat_model("provider:model")`	Returns ChatModel — supports invoke/stream/batch
Define tool	`@tool` decorator	Type hints + docstring mandatory; use `args_schema=` for Pydantic
Add memory	`InMemorySaver()` + thread_id	`{"configurable": {"thread_id": "..."}}` as config arg
Structured output	`response_format=MyModel`	`result["structured_response"]` returns dict or Pydantic instance
RAG tool	`@tool(response_format="content_and_artifact")`	Returns `(str, docs)` tuple
RAG chain	`middleware=[prompt_fn]`	`@dynamic_prompt` or `AgentMiddleware` subclass
Indexing	`Chroma.from_documents(docs, embeddings)`	`persist_directory=` for disk storage
Stream mode	`agent.stream(..., stream_mode=)`	`"values"`, `"messages"`, `"custom"`, or list of multiple
Deploy	`langgraph dev` / `make demo`	Serves at `http://localhost:2024`

CORE MENTAL MODELS

- **create_agent() returns a LangGraph CompiledGraph** — not a simple callable. It has invoke(), stream(), and get_graph() methods.
- **Tools need type hints + docstrings** — the docstring becomes the tool description the LLM reads to decide when to call it.
- **Thread IDs are memory namespaces** — same thread_id = same conversation history; new thread_id = fresh slate.
- **Structured output lives in result["structured_response"]** — TypedDict returns dict, Pydantic returns an instance with attribute access.
- **RAG Agents use 2 LLM calls** (tool call + response); RAG Chains use 1 call (inject context before model).
- **LangGraph dev handles persistence** — no need to pass checkpointer when deploying with langgraph dev.